

Raghavendran V

Full Stack Developer with AI

Session Overview

- Introduction to React
- Create React App using Vite.
- JSX and Transpiling
- Props and JS Expressions
- State and Lifecycle
- React Hooks Deep Dive
- Controlled vs Uncontrolled Components
- React Router

React is a JavaScript library for building user interfaces, particularly for single-page applications (SPAs). It was developed by Facebook and is maintained as an open-source project.

Discuss MVC (Model View Controller) architecture.

◆ **Virtual DOM vs Real DOM**

Real DOM:

- The **actual** Document Object Model (DOM) in the browser.
- **Slow** when you update the UI frequently or on large pages.

Virtual DOM:

- A **lightweight copy** of the real DOM kept in memory.
- React maintains this virtual DOM and **only updates the real DOM where changes are needed.**

How It Works:

1. You change state or props → new virtual DOM created.
2. React **diffs** the old and new virtual DOMs.
3. It **calculates the minimal changes**.
4. Only those changes are applied to the **real DOM** (using efficient updates).

 Result: **Better performance** and smoother user experiences.

◆ Declarative UI

In **declarative programming**, you describe **what the UI should look like**, and React handles **how to update it** when the data changes.

Example (Declarative):

jsx

```
const MyComponent = ({ name }) => <h1>Hello, {name}</h1>;
```

You declare the output for a given state (**name**), and React takes care of rendering and updating the DOM when **name** changes.

vs Imperative Approach (Traditional JS):

```
const h1 = document.createElement("h1");  
h1.innerText = "Hello, " + name;  
document.body.appendChild(h1);
```

Here, you manually manage what and how DOM elements change, which is more error-prone.

◆ Component-Based Architecture

React encourages **building small, reusable pieces of UI** called **components**.

💡 Types of Components:

Function Components (modern way):

```
jsx
```

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

Class Components (older style):

```
jsx
class Welcome extends React.Component {
  render() {
    return <h1>Hello, {this.props.name}</h1>;
  }
}
```

Components can:

- Accept **props** (inputs),
- Manage **state** (internal data),
- Respond to **user events** (like clicks),
- Be **nested** inside each other.

 Think of components like Lego blocks – you can build complex UIs by combining simple blocks.

Summary

Feature	Explanation
React	JS library for building UIs
Declarative UI	Define what UI should look like; React handles updates
Component-based	UIs built from reusable, nested components
Virtual DOM	Fast, memory-based DOM copy used for efficient updates

What is JSX?

JSX (JavaScript XML) is a **syntax extension** for JavaScript used in React to describe UI elements.

It looks like HTML but gets **compiled to JavaScript**.

```
const element = <h1>Hello, World!</h1>;
```

Browsers **cannot understand JSX directly**. JSX must be **transpiled** into standard JavaScript **before** being run in the browser.

Step-by-Step: How Browser Handles JSX

Step 1: Write JSX Code

You write this in your `.jsx` or `.tsx` file:

```
jsx

function Hello() {
  return <h1>Hello from JSX!</h1>;
}
```

Step 2: JSX Transpilation (Using Babel)

Tools like **Babel** transform JSX into regular JavaScript before sending it to the browser.

The JSX above becomes:

js

```
function Hello() {
  return React.createElement('h1', null, 'Hello from JSX!');
}
```

Step 3: Bundling (with Webpack, Vite, etc.)

- Babel is usually part of a build tool like **Webpack**, **Vite**, or **Parcel**.
- These tools also handle bundling your code and resolving imports.

You typically run a command like:

```
npm run build
```

This produces a bundle (like `bundle.js`) which is just **plain JavaScript**, fully understood by browsers.

Step 4: Browser Executes JavaScript

- The browser loads the bundled JS file via `<script src="bundle.js"></script>`.
- React runs in the browser and renders components to the DOM using `ReactDOM.createRoot()` or `ReactDOM.render()`.



Visual Summary:

JSX → (Babel) → React.createElement() JS → (Bundler) → bundle.js → (Browser Executes) → DOM rendered

JSX to Browser: Visual Flow

JSX Code
Written by
Developer

Babel
Transpiles JSX
→ JS

Webpack/Vite
Bundles JS

Browser
Executes JS +
Renders UI

What is Vite?

Vite (pronounced *veet*) is a **modern frontend build tool** that offers:

Key Features:

Feature	Description
 Fast startup	Uses native ES modules in the browser (no bundling initially)
 Hot Module Reload (HMR)	Instant updates while editing
 Built-in bundler	Uses Rollup for production builds
 Minimal config	Works out of the box for many frameworks like React, Vue, etc.
 Lightweight	No heavy Webpack configs or Babel setup needed



How is Vite different from Create React App (CRA)?

Feature	CRA (Create React App)	Vite
Dev Server	Webpack	Native ESM + Fast
Config	Hidden	Transparent & simple
Build Tool	Webpack	Rollup
Speed	Slower startup	Super fast dev
HMR	Slower	Instant updates



How to Create a React App Using Vite

▼ Step 1: Prerequisites

Make sure Node.js (v14+) and npm or yarn is installed:

```
node -v
```

```
npm -v
```



Step 2: Create Your App



Using npm:

```
npm create vite@latest my-react-app -- --template react
```



Using yarn:

```
yarn create vite my-react-app --template react
```

Step 3: Install Dependencies

Navigate into the project and install packages:

```
cd my-react-app
```

```
npm install
```

Step 4: Start the Development Server

```
npm run dev
```

This will start a super-fast Vite dev server at <http://localhost:5173/> by default.

Project Structure:

```
my-react-app/
```

```
├ index.html          ← Entry point (linked to src/main.jsx)
```

```
├ package.json
```

```
├ vite.config.js      ← Vite configuration
```

```
└ src/
```

```
  ├── main.jsx         ← Entry JavaScript file
```

```
  └ App.jsx            ← Root React component
```


How Production Build Works in Vite

For production:

```
npm run build
```

This uses **Rollup** to bundle your app efficiently and outputs static assets to the `dist/` folder.

To preview your production build:

```
npm run preview
```

Summary

Step	Command
Create project	<pre>npm create vite@latest my-app -- --template react</pre>
Install dependencies	<pre>npm install</pre>
Run dev server	<pre>npm run dev</pre>
Build for production	<pre>npm run build</pre>

◆ What is State in React?

State is **data that changes over time** within a component.

When state changes, **React re-renders** the component to reflect the new data.

✓ Characteristics:

- Managed **within** the component
- Can be changed using `useState` (in functional components)
- Triggers re-render when updated

◆ What are Props in React?

Props (short for **properties**) are used to **pass data from a parent to a child component**.

✓ Characteristics:

- Read-only
- Passed as **attributes** on components
- Cannot be changed by the child



State vs Props: Key Differences

Feature	Props	State
Definition	Data passed to components	Data managed within component
Read/Write	Read-only	Can change (mutable)
Source	Parent component	Own component
Purpose	Pass data & behavior to children	Track and manage internal changes
Lifecycle	Does not change in component	Can change over time
Usage	<code><Child title=\"Hello\" /></code>	<code>const [count, setCount] = useState(0)</code>

When to Use Props vs State

-  **Use Props** when:
 - Passing fixed or dynamic data from parent
 - Making components reusable
-  **Use State** when:
 - Tracking local changes like user input, API data, toggles, etc.

♦ What is a Pure Component in React?

A **Pure Component** is a React component that **renders the same output** for the **same props and state**. It implements a **shallow comparison** to avoid unnecessary re-renders.

✓ In class components, React provides `React.PureComponent`.

✓ In function components, we can achieve the same using `React.memo()`.

🧠 Why Use Pure Components?

Pure Components are used for **performance optimization**.

They prevent re-renders if the **props or state haven't changed** — saving time and improving app speed.

🔧 Example: Class-Based Pure Component

```
jsx
CopyEdit
import React, { PureComponent } from 'react';

class MyComponent extends PureComponent {
  render() {
    console.log("Rendered");
    return <h1>Hello, {this.props.name}</h1>;
  }
}
```

Functional Equivalent: **React.memo()**

jsx

```
const MyComponent = React.memo(function MyComponent({ name }) {  
  console.log("Rendered");  
  return <h1>Hello, {name}</h1>;  
});
```

React.memo() is a Higher-Order Component:

It wraps your function component and prevents re-rendering unless the props have changed.

Shallow Comparison (Important!)

Pure components do a **shallow comparison**:

- Primitives (string, number, boolean) are compared by value.
- Objects/arrays/functions are compared by **reference**, not content.

js

```
{ a: 1 } === { a: 1 } // false (different references)
```

 So if you pass objects/arrays as props, PureComponent might still re-render unless you're careful with immutability.

When to Use PureComponent / React.memo

Use Case	Recommended?
Components re-rendering too often	 Yes
Props/state rarely change	 Yes
Performance bottleneck in large app	 Yes
Props are always changing or complex	 No

Normal Component vs Pure Component

Feature	Normal Component	Pure Component
Re-render	Always on state/prop update	Only if props/state change (shallow)
Comparison logic	None	Shallow comparison
Performance	Less optimized	More optimized



What is the React Component Lifecycle?

The **React lifecycle** represents the **stages a component goes through** from creation to destruction:



Lifecycle Phases:

1. **Mounting** – Component is being created and inserted into the DOM.
2. **Updating** – Component is re-rendered due to state or props change.
3. **Unmounting** – Component is removed from the DOM.
4. **Error Handling** – Errors caught during rendering or lifecycle methods.



1. Mounting Phase

Method (Class)

Hook (Functional)

`constructor()`

`useState()`

`static getDerivedStateFromProps()`

-

`render()`

JSX return

`componentDidMount()`

`useEffect(() => {}, [])`



2. Updating Phase

Triggered by `setState()` or new props.

Method (Class)

Hook (Functional)

`static getDerivedStateFromProps()`

-

`shouldComponentUpdate()`

`React.memo()`

`render()`

JSX return

✗ 3. Unmounting Phase

Example:

jsx

CopyEdit

```
useEffect(() => {  
  return () => {  
    console.log("Component unmounted");  
  };  
}, []);
```

Method (Class)

Hook (Functional)

`componentWillUnmount()`

`useEffect(() => { return () => {} }, [])`

⚡ Class vs Functional Lifecycle Comparison

Phase	Class Method	Functional Hook
Mount	<code>componentDidMount()</code>	<code>useEffect(() => {}, [])</code>
Update	<code>componentDidUpdate()</code>	<code>useEffect(() => {}, [dep])</code>
Unmount	<code>componentWillUnmount()</code>	<code>useEffect(() => { return () => {} }, [])</code>

♦ What is React Router?

React Router is a **standard routing library** for React applications. It allows you to:

- ✓ Create **single-page applications (SPAs)**
- ✓ Navigate between **different pages or views**
- ✓ Load components based on the **URL path**
- ✓ Handle **dynamic routing**, **nested routes**, and **protected routes**

↺ Why Do We Need React Router?

In traditional websites, the browser loads a **new HTML page** for every route change.

In React, we build **SPAs**, so the **page doesn't reload**. React Router enables **dynamic component rendering** on route change without reloading the page.

📦 Packages:

Package	Purpose
<code>react-router</code>	Core routing logic
<code>react-router-dom</code>	For React apps in browsers ✓
<code>react-router-native</code>	For mobile apps with React Native